Date:06/10/2026

7. Addressing underfitting and overfitting through validation curve

Procedure:

Step 1: Import the required libraries.

Step 2: Load the breast cancer dataset.

Step 3: Separate the dataset into input variables X and output variable y.

Step 4: Create a polynomial logistic regression model.

Step 5: Set the polynomial degrees from 1 to 5.

Step 6: Apply the validation curve using 5-fold cross-validation.

Step 7: Calculate the mean training and validation accuracy.

Step 8: Plot the training accuracy and validation accuracy against polynomial degree.

Step 9: Observe the graph to identify underfitting and overfitting.

Step 10: Select the model with the best validation accuracy.

Solution:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import learning_curve
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
import pandas as pd

df = pd.read_csv(r"C:\Users\Lenovo\Downloads\KAVYA SDM College(Autonomous), Ujire - wdbc.data",
header=None)
print(df.head())

from sklearn.preprocessing import LabelEncoder

X = df.loc[:, 2:].values
y = df.loc[:, 1].values

le = LabelEncoder()
y = le.fit_transform(y)

le.transform(['M', 'B'])

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
```

```
    y,
    test_size=0.20,
    random_state=1
)

pipe_lr = Pipeline([
    ('scl', StandardScaler()),
    ('clf', LogisticRegression(penalty='l2', random_state=0, max_iter=10000))
])

train_sizes, train_scores, test_scores = learning_curve(
    estimator=pipe_lr,
    X=X_train,
    y=y_train,
    train_sizes=np.linspace(0.1, 1.0, 10),
    cv=10,
    n_jobs=1
)

train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.plot(
    train_sizes,
    train_mean,
    color='blue',
    marker='o',
    markersize=5,
    label='training accuracy'
)

plt.fill_between(
    train_sizes,
    train_mean + train_std,
    train_mean - train_std,
    alpha=0.15,
    color='blue'
)

# Test Score
plt.plot(
    train_sizes,
    test_mean,
    color='green',
    linestyle='--',
    marker='s',
```

```
    markersize=5,
    label='validation accuracy'
)

plt.fill_between(
    train_sizes,
    test_mean + test_std,
    test_mean - test_std,
    alpha=0.15,
    color='green'
)

plt.grid()
plt.xlabel('Number of training samples')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.ylim([0.8, 1.0])
plt.show()

from sklearn.model_selection import validation_curve

param_range = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]

train_scores, test_scores = validation_curve(
    estimator=pipe_lr,
    X=X_train,
    y=y_train,
    param_name='clf__C',
    param_range=param_range,
    cv=10
)

train_mean = np.mean(train_scores, axis=1)
train_std = np.std(train_scores, axis=1)
test_mean = np.mean(test_scores, axis=1)
test_std = np.std(test_scores, axis=1)

plt.plot(
    param_range,
    train_mean,
    color='blue',
    marker='o',
    markersize=5,
    label='training accuracy'
)

plt.fill_between(
    param_range,
```

```
    train_mean + train_std,
    train_mean - train_std,
    alpha=0.15,
    color='blue'
)

plt.plot(
    param_range,
    test_mean,
    color='green',
    linestyle='--',
    marker='s',
    markersize=5,
    label='validation accuracy'
)

plt.fill_between(
    param_range,
    test_mean + test_std,
    test_mean - test_std,
    alpha=0.15,
    color='green'
)

plt.grid()
plt.xscale('log')
plt.legend(loc='lower right')
plt.xlabel('Parameter C')
plt.ylabel('Accuracy')
plt.ylim([0.8, 1.0])
plt.show()
```

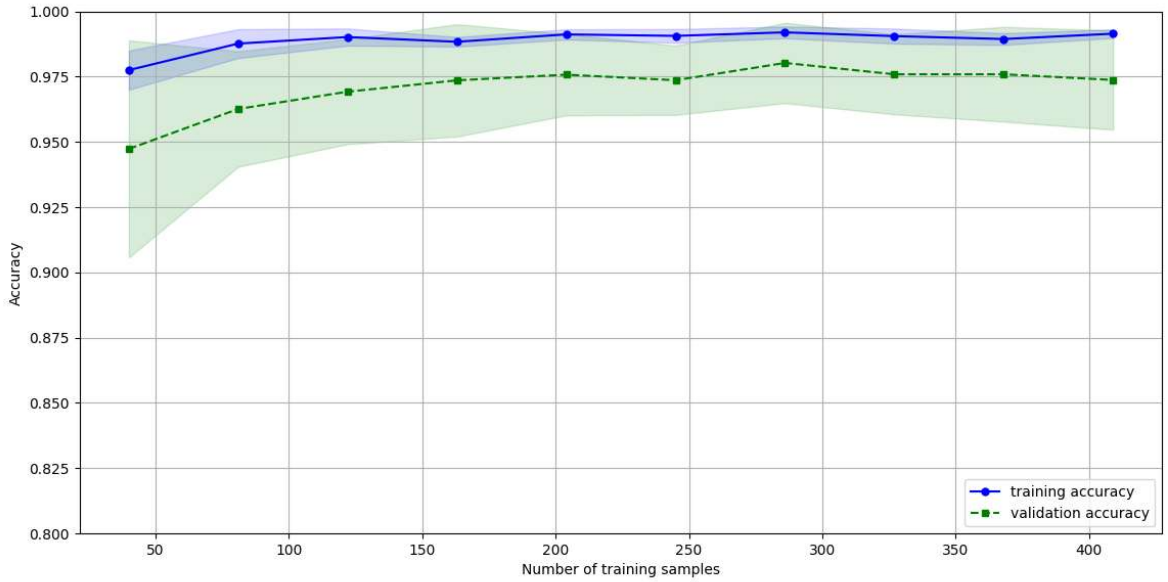
0 842302 M 17.99 10.38 122.80 1001.0 0.11840 ... 2019.0 0.1622 0.6656 0.7119 0.2654 0.4601 0.11890

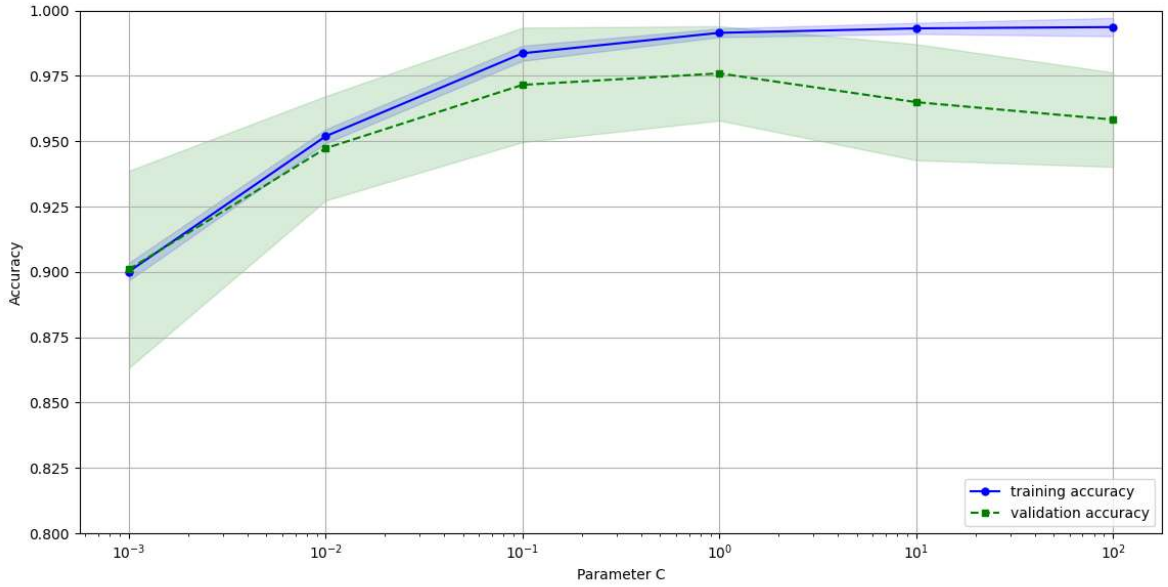
1 842517 M 20.57 17.77 132.90 1326.0 0.08474 ... 1956.0 0.1238 0.1866 0.2416 0.1860 0.2750 0.08902

2 84300903 M 19.69 21.25 130.00 1203.0 0.10960 ... 1709.0 0.1444 0.4245 0.4504 0.2430 0.3613 0.08758

3 84348301 M 11.42 20.38 77.58 386.1 0.14250 ... 567.7 0.2098 0.8663 0.6869 0.2575 0.6638 0.17300

4 84358402 M 20.29 14.34 135.10 1297.0 0.10030 ... 1575.0 0.1374 0.2050 0.4000 0.1625 0.2364 0.07678


1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.800
Accuracy
50
100
150
200
250
300
350
400
Number of training samples
training accuracy
validation accuracy


1.000
0.975
0.950
0.925
0.900
0.875
0.850
0.825
0.800
Accuracy
10^{-3}
10^{-2}
10^{-1}
10^{0}
10^{1}
10^{2}
Parameter C
training accuracy
validation accuracy